Ecole Nationale des Sciences Appliquées

Année universitaire 2025-2026

GI : S8

# Projet : Conception et réalisation d'un système de surveillance sécurisée d'un répertoire sous Linux

## 1. Intitulé du projet

**Conception et réalisation d'un système de surveillance sécurisée d'un répertoire sous Linux**

## 2. Contexte

*Dans un environnement Linux, certains répertoires contiennent des fichiers sensibles ou importants, tels que des documents administratifs, des données de configuration, des journaux, des codes sources ou des fichiers applicatifs. Toute modification non autorisée, suppression accidentelle ou création de fichiers suspects peut compromettre l'intégrité du système ou des données.*

*Dans ce contexte, il est utile de disposer d'un outil capable de surveiller en temps réel l'activité d'un répertoire donné, d'enregistrer les événements détectés et de signaler les comportements anormaux. Ce projet s'inscrit dans le domaine de la programmation système et vise à mettre en œuvre plusieurs notions fondamentales de Linux, notamment la gestion des fichiers, les processus, les threads, les signaux, la communication inter-processus et la supervision d'événements.*

## 3. Problématique

*Comment concevoir une application système sous Linux capable de surveiller un répertoire en temps réel, d'enregistrer les actions effectuées sur les fichiers et sous-répertoires, et d'identifier certaines activités suspectes tout en garantissant un fonctionnement fiable, structuré et sécurisé ?*

## 4. Objectif général

*L'objectif principal de ce projet est de développer une application en langage C sous Linux permettant de :*

- *surveiller un répertoire donné en temps réel ;*
- *détecter les principaux événements liés aux fichiers et répertoires ;*
- *journaliser les événements dans un fichier log ;*
- *afficher des alertes en cas d'activité suspecte ;*

- *produire des statistiques d'exécution ;*
- *assurer un arrêt propre du système en libérant toutes les ressources.*

## 5. Objectifs spécifiques

*Le projet doit permettre à l'étudiant de maîtriser les points suivants :*

- *manipulation des fichiers et des répertoires ;*
- *utilisation des appels système Linux ;*
- *création et synchronisation de processus ou de threads ;*
- *mise en œuvre d'un mécanisme IPC ;*
- *gestion des signaux système ;*
- *traitement d'événements en temps réel ;*
- *journalisation et traçabilité ;*
- *détection simple d'anomalies ou d'activités suspectes.*

## 6. Description générale de la solution

*L'application à réaliser est un outil de surveillance locale qui s'exécute en ligne de commande. L'utilisateur fournit en argument le chemin du répertoire à surveiller. Le système observe en continu les modifications sur ce répertoire et effectue les tâches suivantes :*

- *détecter les créations, suppressions, modifications et renommages ;*
- *enregistrer chaque événement dans un journal ;*
- *analyser les événements pour repérer des comportements suspects ;*
- *notifier l'utilisateur par affichage console ou message d'alerte ;*
- *permettre l'arrêt du programme de façon propre via un signal ou une commande.*

Exemple d'exécution :

```
./securebox /home/etudiant/documents
```

## 7. Périmètre du projet

### 7.1 Inclus dans le projet

*Le projet couvre :*

- *la surveillance d'un répertoire local sous Linux ;*
- *la détection d'événements sur les fichiers et sous-répertoires ;*
- *la journalisation des événements détectés ;*
- *la génération d'alertes simples ;*

- *l'affichage de statistiques ;*
- *l'utilisation des mécanismes systèmes appropriés.*

### 7.2 Hors périmètre

*Ne sont pas exigés dans la version de base :*

- *interface graphique ;*
- *surveillance distante via réseau ;*
- *authentification avancée des utilisateurs ;*
- *chiffrement complexe des fichiers surveillés ;*
- *intelligence artificielle pour la détection avancée d'anomalies ;*
- *intégration cloud.*

## 8. Utilisateurs visés

*Les utilisateurs cibles sont :*

- *administrateurs système ;*
- *étudiants en programmation système ;*
- *enseignants souhaitant illustrer les concepts Linux ;*
- *utilisateurs souhaitant protéger un répertoire local sensible.*

## 9. Besoins fonctionnels

*Le système doit répondre aux besoins suivants :*

- *surveiller un répertoire fourni par l'utilisateur ;*
- *détecter les événements système significatifs ;*
- *mémoriser les événements dans un fichier journal ;*
- *distinguer les événements normaux des événements suspects ;*
- *signaler les anomalies ;*
- *permettre la consultation de statistiques ;*
- *gérer correctement les erreurs d'exécution ;*
- *s'arrêter proprement sans laisser de ressources ouvertes.*

## 10. Exigences fonctionnelles détaillées

### 10.1 Lancement de l'application

*Le programme doit être lancé en ligne de commande avec le chemin du répertoire à surveiller.*

### *Exigences :*

- *le chemin doit être obligatoire ;*
- *le programme doit vérifier l'existence du répertoire ;*
- *le programme doit vérifier que le chemin fourni correspond bien à un répertoire ;*
- *le programme doit vérifier les droits d'accès nécessaires ;*
- *en cas d'erreur, un message explicite doit être affiché.*

### *Exemple :*

```
./securebox /home/user/test
```

## 10.2 Surveillance du répertoire

*Le système doit surveiller le répertoire cible en temps réel.*

### *Événements minimaux à détecter :*

- *création de fichier ;*
- *suppression de fichier ;*
- *modification de fichier ;*
- *renommage de fichier ;*
- *création de sous-répertoire ;*
- *suppression de sous-répertoire ;*
- *déplacement interne si détectable.*

### *Attendus :*

- *les événements doivent être capturés dès leur apparition ;*
- *le mécanisme de surveillance doit fonctionner tant que le programme est actif ;*
- *aucune intervention manuelle ne doit être nécessaire après le lancement.*

## 10.3 Journalisation

*Chaque événement détecté doit être enregistré dans un fichier journal.*

### *Informations minimales à enregistrer :*

- *date ;*
- *heure ;*
- *type d'événement ;*
- *nom de la ressource concernée ;*

- *chemin du fichier ou du répertoire ;*
- *niveau de gravité éventuel ;*
- *identifiant du processus de surveillance, si pertinent.*

### *Format attendu du log :*

```
[2026-04-23 14:12:10] [INFO] CREATE_FILE : /home/user/test/data.txt
[2026-04-23 14:12:25] [INFO] MODIFY_FILE : /home/user/test/report.pdf
[2026-04-23 14:12:30] [ALERTE] DELETE_FILE : /home/user/test/notes.txt
```

### *Contraintes :*

- *le fichier log doit être créé automatiquement s'il n'existe pas ;*
- *les écritures doivent être fiables ;*
- *les messages doivent être lisibles et ordonnés ;*
- *les logs ne doivent pas être perdus lors de l'arrêt.*

## 10.4 Analyse de sécurité

*Le système doit être capable de repérer certaines activités suspectes à partir des événements collectés.*

### *Situations considérées comme suspectes :*

- *suppression répétée de plusieurs fichiers dans un court intervalle ;*
- *modifications multiples rapides sur un même fichier ;*
- *apparition d'un fichier caché inattendu ;*
- *création d'un fichier avec extension interdite ;*
- *renommage anormal d'un fichier sensible ;*
- *création ou déplacement d'un script exécutable dans le répertoire surveillé.*

### *Exemples de règles :*

- *plus de 5 suppressions en moins de 10 secondes ;*
- *plus de 3 modifications d'un même fichier en moins de 5 secondes ;*
- *création d'un fichier commençant par . ;*
- *détection d'une extension appartenant à une liste noire.*

## 10.5 Génération d'alertes

*En cas d'activité suspecte, le système doit produire une alerte.*

### *Formes possibles :*

- *affichage console ;*
- *écriture spéciale dans le fichier log ;*
- *émission d'un signal interne ;*
- *envoi dans un tube ou une file de messages à un module d'alerte.*

### Exemple :

```
[ALERTE] Activité suspecte détectée : 6 suppressions en 8 secondes dans
/home/user/test
```

## 10.6 Statistiques

*Le système doit maintenir des statistiques sur son activité.*

### *Statistiques minimales :*

- *nombre total d'événements ;*
- *nombre de créations ;*
- *nombre de suppressions ;*
- *nombre de modifications ;*
- *nombre de renommages ;*
- *nombre d'alertes ;*
- *durée de fonctionnement.*

### *Consultation :*

- *à la réception d'un signal prédéfini ;*
- *ou à la fermeture du programme ;*
- *ou via une commande utilisateur si cette option est retenue.*

## 10.7 Arrêt propre

Le système doit pouvoir s'arrêter proprement.

### Le programme doit :

- *intercepter certains signaux ;*
- *fermer les fichiers ouverts ;*
- *libérer la mémoire allouée ;*
- *fermer les canaux IPC ;*
- *arrêter les threads ou attendre la fin des processus fils ;*

- *enregistrer un message de fin dans le journal.*

## 11. Exigences non fonctionnelles

### 11.1 Performance

- *le système doit fonctionner avec un temps de réponse raisonnable ;*
- *la surveillance ne doit pas engendrer une forte consommation CPU ;*
- *les traitements doivent rester fluides même en cas d'événements fréquents.*

### 11.2 Fiabilité

- *le système ne doit pas se bloquer lors d'un événement inattendu ;*
- *les erreurs doivent être détectées et traitées ;*
- *le système doit rester stable pendant une exécution prolongée.*

### 11.3 Maintenabilité

- *le code doit être modulaire ;*
- *les fonctions doivent être bien séparées ;*
- *les noms de variables et fonctions doivent être explicites ;*
- *les commentaires doivent être suffisants.*

### 11.4 Portabilité

- *l'application est destinée principalement à Linux ;*
- *elle doit être compilable avec gcc sur une distribution Linux classique.*

### 11.5 Sécurité

- *l'application ne doit pas exécuter de commandes dangereuses non contrôlées ;*
- *elle ne doit pas utiliser system() pour les opérations principales ;*
- *elle doit contrôler les accès aux fichiers créés.*

## 12. Architecture fonctionnelle recommandée

*Une architecture modulaire est recommandée.*

### Module 1 : Initialisation

*Responsable de :*

- *lire les arguments ;*

- *vérifier le répertoire cible ;*
- *initialiser les ressources ;*
- *préparer le fichier log.*

### Module 2 : Surveillance

*Responsable de :*

- *écouter les événements système ;*
- *récupérer les notifications de changement ;*
- *transmettre les événements au module d'analyse.*

### Module 3 : Analyse

*Responsable de :*

- *interpréter les événements ;*
- *déterminer leur nature ;*
- *vérifier s'ils correspondent à une règle suspecte ;*
- *produire une décision.*

### Module 4 : Journalisation

*Responsable de :*

- *recevoir les événements validés ;*
- *écrire les informations dans le log ;*
- *garantir la cohérence des écritures.*

### Module 5 : Alerte

*Responsable de :*

- *afficher les alertes ;*
- *les consigner ;*
- *transmettre éventuellement l'information à d'autres modules.*

### Module 6 : Statistiques

*Responsable de :*

- *mettre à jour les compteurs ;*
- *préparer un résumé en temps réel ou à la demande.*

## 13. Architecture technique recommandée

# *Option 1 : Architecture multi-processus*

- ***Processus principal** : initialisation et coordination ;*
- ***Processus de surveillance** : capture des événements ;*
- ***Processus de journalisation** : écrit dans le log ;*
- ***Processus d'alerte/analyse** : traite les événements suspects.*

***IPC possibles :***

- *pipe ;*
- *FIFO ;*
- *file de messages ;*
- *mémoire partagée avec sémaphores.*

# *Option 2 : Architecture multi-threads*

- ***Thread de surveillance** ;*
- ***Thread d'analyse** ;*
- ***Thread de journalisation** ;*
- ***Thread principal** pour la gestion globale.*

***Synchronisation :***

- *mutex ;*
- *sémaphores ;*
- *variables de condition.*

## 14. Contraintes techniques

### 14.1 Langage

- *le projet doit être réalisé en **langage C**.*

### 14.2 Système d'exploitation

- *le projet doit être exécuté sous **Linux**.*

### 14.3 Outils autorisés

- `gcc`

- `make`
- `gdb`
- `valgrind`
- *bibliothèques standards de C*
- *bibliothèques POSIX*
- *mécanisme inotify sous Linux pour la surveillance*

### 14.4 Appels système et fonctions possibles

- `open()`, `close()`, `read()`, `write()`
- `stat()`, `lstat()`
- `opendir()`, `readdir()`
- `fork()`, `wait()`, `waitpid()`
- `pipe()`, `mkfifo()`
- `shmget()`, `shmat()`
- `msgget()`, `msgsnd()`, `msgrcv()`
- `signal()`, `sigaction()`
- `pthread_create()`, `pthread_join()`
- `inotify_init()`, `inotify_add_watch()`, `inotify_rm_watch()`

### 14.5 Interdictions

- *usage de system() pour contourner les appels système ;*
- *code monolithique non structuré ;*
- *absence de gestion d'erreurs ;*
- *dépendance obligatoire à une interface graphique.*

## 15. Interfaces d'entrée et de sortie

### 15.1 Entrées

- *chemin du répertoire à surveiller ;*
- *éventuelles options de configuration ;*
- *signaux système ;*
- *événements du système de fichiers.*

### 15.2 Sorties

- *affichage console ;*
- *fichier journal ;*
- *statistiques finales ;*
- *messages d'alerte.*

## 16. Structure proposée du projet

```
securebox/
│
├── main.c
├── monitor.c
├── monitor.h
├── logger.c
├── logger.h
├── analyzer.c
├── analyzer.h
├── stats.c
├── stats.h
├── signals.c
├── signals.h
├── utils.c
├── utils.h
├── config.h
├── Makefile
├── README.md
└── logs/
    └── securebox.log
```

## 17. Scénario de fonctionnement attendu

1. *L'utilisateur lance le programme avec le chemin du répertoire.*
2. *Le programme vérifie le chemin et initialise les ressources.*
3. *Le module de surveillance démarre l'écoute des événements.*
4. *Chaque événement capturé est transmis à l'analyse.*
5. *L'événement est classé comme normal ou suspect.*
6. *Il est enregistré dans le log.*
7. *Si l'événement est suspect, une alerte est affichée.*
8. *Les statistiques sont mises à jour.*
9. *À l'arrêt, un résumé final est affiché et enregistré.*

## 18. Livrables attendus

*Les étudiants doivent fournir :*

- *le code source complet ;*
- *les fichiers d'en-tête ;*
- *un Makefile fonctionnel ;*
- *un rapport technique ;*
- *un manuel d'utilisation ou README ;*
- *des scénarios de test ;*
- *éventuellement une démonstration.*

## 19. Contenu attendu du rapport

- *Le rapport doit contenir au minimum :*
- *introduction ;*

- *contexte et problématique* ;
- *objectifs ;*
- *analyse des besoins ;*
- *architecture choisie ;*
- *description des modules ;*
- *appels système utilisés ;*
- *jeux de tests ;*
- *résultats obtenus ;*
- *difficultés rencontrées ;*
- *conclusion et perspectives.*

## 20. Plan de réalisation recommandé

### Phase 1 : Analyse

- *compréhension du besoin ;*
- *définition des fonctionnalités ;*
- *choix d'architecture.*

### Phase 2 : Conception

- *découpage modulaire ;*
- *définition des structures de données ;*
- *choix du mécanisme IPC.*

### Phase 3 : Implémentation de base

- *initialisation ;*
- *surveillance du répertoire ;*
- *affichage simple des événements.*

### Phase 4 : Journalisation

- *création du fichier log ;*
- *écriture structurée des événements.*

### Phase 5 : Détection d'alertes

- *implémentation des règles simples ;*
- *émission d'alertes.*

### Phase 6 : Signaux et terminaison

- *gestion de SIGINT, SIGTERM, SIGUSR1 ;*
- *arrêt propre.*

### Phase 7 : Tests et validation

- *tests unitaires ;*
- *tests d'intégration ;*
- *cas normaux et cas d'erreur.*

### 21. Jeux de tests minimaux

## *Test 1*

*Lancement avec un répertoire valide.*

## *Test 2*

*Lancement avec un chemin inexistant.*

## *Test 3*

*Création d'un fichier dans le répertoire surveillé.*

## *Test 4*

*Modification d'un fichier existant.*

## *Test 5*

*Suppression d'un fichier.*

## *Test 6*

*Création d'un fichier caché.*

## *Test 7*

*Création d'un fichier avec extension interdite.*

## *Test 8*

*Suppressions répétées en peu de temps.*

### *Test 9*

*Interruption du programme par Ctrl+C.*

### *Test 10*

*Affichage des statistiques à la réception d'un signal.*

## 22. Critères de validation

*Le projet sera considéré conforme si :*

- *le programme compile sans erreurs majeures ;*
- *la surveillance fonctionne correctement ;*
- *les événements sont bien détectés ;*
- *le log est correctement généré ;*
- *les alertes sont cohérentes ;*
- *les ressources sont libérées à l'arrêt ;*
- *le code est structuré et lisible ;*
- *le rapport est clair et complet.*

## 24. Extensions possibles

*Pour enrichir le projet, les extensions suivantes peuvent être envisagées :*

- *surveillance récursive des sous-répertoires ;*
- *quarantaine automatique des fichiers suspects ;*
- *sauvegarde automatique avant modification ;*
- *export des logs en CSV ;*
- *interface en mode texte avec ncurses ;*
- *chiffrement du fichier journal ;*
- *filtrage par type d'événement ;*
- *consultation des logs par date ou gravité.*